Bounded-Staleness Reconciliation for Batched Settlement Ledgers

Ines Valcourt[1] · Oren Mathiesen[2] · Priyamvada Lenkov[1] · Callum Ardery[2]

[1]*Laboratory for Distributed Ledgers, Northfold Institute of Technology* [2]*Meridian Payments, Settlement Research Group*
{ivalcourt, plenkov}@northfold.example.com {omathiesen, cardery}@meridian.example.com

Abstract. Settlement platforms commit transactions to an append-only log long before those transactions become visible to the balances that readers consult. The gap between the two — *staleness* — is usually governed by a fixed batch window chosen once and left alone. We show that the fixed window is the wrong abstraction: the staleness-minimising window depends on load, on per-batch overhead and on per-item reconciliation cost, all of which drift over a working day. We derive a closed form for the optimal window under a batched M/G/1 model, prove that expected staleness is convex in the window above the stability threshold, and give a Chernoff-style tail bound for the batch service time. These results motivate Tidewater, a plug-in controller that estimates the model online with recursive least squares and steers the window toward the analytic optimum. On a fourteen-day replay of a sandbox settlement workload, Tidewater reduces 99th-percentile staleness by 41–78% relative to the best fixed window at every load we tested, while holding throughput within 1.2% of an offline oracle.

Keywords — settlement ledgers, batching, staleness, queueing control, recursive least squares

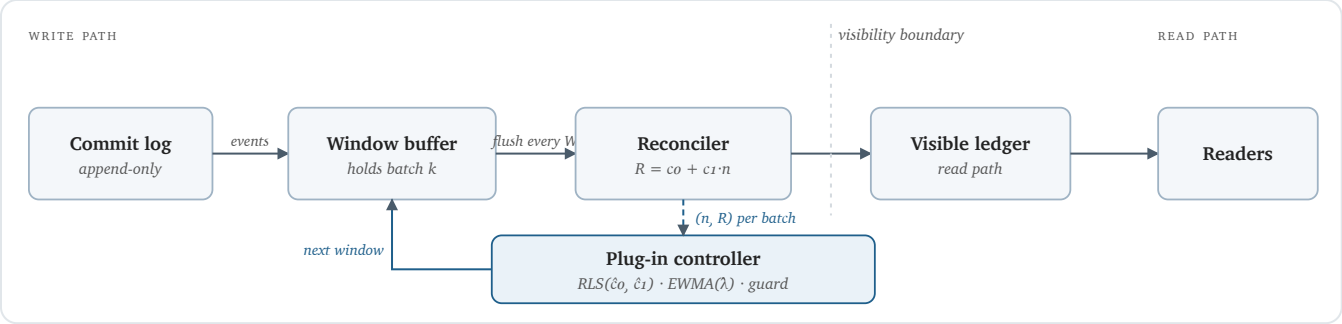


Figure 1. The reconciliation pipeline. Committed events accumulate in a window buffer that is flushed every W_k milliseconds; the reconciler folds each batch into the visible ledger. Tidewater (blue) observes each batch's size and service time and chooses the next window. Everything left of the dashed boundary is invisible to readers.

1 Introduction

A payment that has been accepted is not yet a payment that can be seen. Between the moment an event is durably appended to a settlement log and the moment its effect appears in the balances that downstream readers query, there is an interval during which the system knows something its readers do not. Operators call this interval *reconciliation lag*; we call it staleness, and we argue that it deserves the same first-class treatment as throughput and durability.

Almost every production ledger we are aware of reconciles in batches. Batching amortises a fixed per-batch cost — opening a transaction against the balance store, acquiring range locks, emitting an audit record — across many events. The batch is usually closed by a timer, and the timer's period is a configuration constant. Pick it too short and the fixed cost dominates, the reconciler saturates, and a queue forms in front of it. Pick it too long and every event waits, on average, half a window before it is even considered. Between the two lies a minimum, and the central observation of this paper is that the minimum moves.

It moves because its location depends on three quantities that are not constant: the arrival rate λ, the per-batch overhead c_0, and the per-event cost c_1. Arrival rates follow the working day; overhead climbs when the balance store is under compaction; per-event cost rises when a larger fraction of events touch contended accounts. A window that was optimal at nine in the morn-

ing can be twice too long by noon and dangerously short at the end-of-day cut-over.[1]

We make three contributions. First (§3), we model the reconciler as a batched single-server queue and derive the expected staleness as a function of the window. The resulting objective is convex on its domain and has a closed-form minimiser that is pleasant enough to compute on every batch. Second (§4), we describe Tidewater, a controller that estimates the model parameters online and plugs them into the closed form, with a guard that keeps the system stable when the estimates are poor. Third (§5), we evaluate Tidewater on a replayed workload and isolate the contribution of each component.

2 Background

2.1 Ledgers, logs and visibility

We consider a ledger split into a *commit log*, to which writers append events, and a *visible ledger*, from which readers obtain balances. An event i is committed at time t_i^{c} and becomes visible at t_i^{v}. Its staleness is

$$S_i \;=\; t_i^{\mathrm{v}} - t_i^{\mathrm{c}} \;\geq\; 0, \tag{1}$$

and we are interested both in the mean $\mathbb{E}[S]$ and in high quantiles such as $S_{0.99}$. The commit log is the source of truth: staleness never threatens durability, only the timeliness of what readers see. That distinction is what makes batching acceptable in the first place.

2.2 Why not reconcile every event?

Per-event reconciliation drives staleness toward the bare service time, but only if the reconciler can keep up. With a per-batch overhead of a few milliseconds and arrival rates in the hundreds of thousands of events per second, it cannot. The question is never *whether* to batch but *how long* to wait before closing a batch, and that is the question this paper answers.

2.3 Existing policies

Three policies dominate practice. A *fixed window* closes the batch every W milliseconds. A *size trigger* closes it when n events have accumulated, sometimes with a timer as a fallback. A *hybrid* does whichever comes first. All three expose a constant that someone must choose, and none of them observes the reconciler's own cost. The adaptive schemes in the literature [4, 9, 13] adjust the window from observed latency alone, which — as §5.4 shows — makes them oscillate when overhead and load change together.

3 Model

Events arrive as a Poisson process of rate λ. The window buffer is flushed every W time units, so batch k contains $N_k \sim \mathrm{Poisson}(\lambda W)$ events. Reconciling a batch of n events takes

$$R(n) \;=\; c_0 + c_1 n, \tag{2}$$

where c_0 is the per-batch overhead and c_1 the per-event cost. We write $\rho_1 = c_1\lambda$ for the load due to per-event work alone and $u = 1 - \rho_1$ for the capacity that remains. Batches are offered to the reconciler once per window, so its utilisation is

$$\rho(W) \;=\; \frac{\mathbb{E}[R(N)]}{W} \;=\; \frac{c_0}{W} + \rho_1. \tag{3}$$

The system is stable if and only if $\rho(W) < 1$, that is, $W > W_{\min} := c_0/u$. Below that threshold the overhead alone outruns the clock.

3.1 Expected staleness

An event waits for its window to close, then for the batch ahead of it to finish, then for its own batch to be reconciled. Under Poisson arrivals the first term has mean $W/2$. Approximating the remaining delay by the mean response time of a single-server queue at utilisation $\rho(W)$ gives

$$\mathbb{E}[S](W) \;=\; \frac{W}{2} \;+\; \frac{c_0 + \rho_1 W}{1 - \rho(W)}. \tag{4}$$

The approximation is deliberately simple: it ignores the variance of N_k, which is small relative to its mean at the batch sizes we consider (λW in the thousands), and §5.2 shows that it tracks measurement to within six percent across the stable range.

Lemma 1 (convexity). *On $W > W_{\min}$, $\mathbb{E}[S](W)$ is strictly convex.*

Proof sketch. Substitute $z = uW - c_0 > 0$. Using $\rho_1 + u = 1$, the second term of (4) becomes $\frac{\rho_1}{u^2} z + \frac{c_0^2}{u^2}\, z^{-1} +$ const, which is affine in z plus a positive multiple of $1/z$. Both $W/2$ and z are affine in W, and $1/z$ is strictly convex for $z > 0$. ∎

Proposition 2 (optimal window). *The unique minimiser of (4) is*

$$W^\star \;=\; \frac{c_0}{1-\rho_1}\left(1+\sqrt{\frac{2}{1+\rho_1}}\right). \tag{5}$$

Differentiating the form in the proof of Lemma 1 and setting the result to zero gives $z^{\star 2} = 2c_0^2/(1+\rho_1)$, from which (5) follows. Two features of (5) matter in practice. The optimum scales *linearly* in the overhead c_0, so a store that doubles its per-batch cost during compaction wants a window twice as long. And it grows without bound as $\rho_1 \to 1$: near saturation, the right response is to batch more, not less — the opposite of what a latency-only controller does.

For the workload of §5 ($c_0 = 4$ ms, $c_1 = 3.5\,\mu$s, $\lambda = 100$ events/ms) we have $\rho_1 = 0.35$ and $W^\star \approx 13.6$ ms, against a stability threshold of $W_{\min} \approx 6.2$ ms. Fig. 2 plots (4) at three loads.

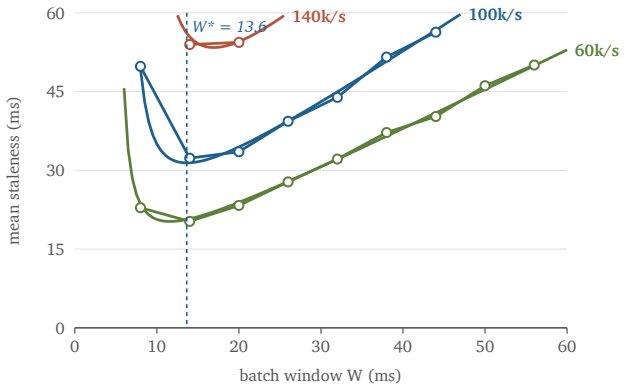


Figure 2. Expected staleness against the window at three arrival rates: equation (4) as lines, measured means as circles. The dashed line marks W^* at 100k events/s. Each curve rises steeply toward its stability threshold on the left and linearly on the right.

3.2 Tail behaviour

Means are not what operators are paged for. To bound the tail we bound the service time of a single batch. Since N_k is Poisson with mean $\mu = \lambda W$, the standard multiplicative Chernoff bound gives, for $\delta > 0$,

$$\Pr\!\left[N_k \ge (1+\delta)\mu\right] \;\le\; \exp\!\left(-\frac{\delta^2 \mu}{2+\delta}\right), \tag{6}$$

and therefore $\Pr[R(N_k) \ge c_0 + c_1(1+\delta)\mu]$ obeys the same bound. At $\mu = 1{,}364$ the probability that a batch exceeds its mean size by 10% is below 1.5×10^{-3}; by 20%, below 2.6×10^{-9}. Batch-size variance is therefore not what produces tail staleness. The tail is produced by drift in c_0 and λ — which is exactly what a fixed window cannot respond to.

4 The Tidewater controller

Proposition 2 turns window selection into parameter estimation. Tidewater maintains estimates $\hat c_0$, $\hat c_1$ and $\hat\lambda$, computes $W^\star$ from them, and moves the window toward it at a bounded rate. Algorithm 1 gives the complete procedure; it runs once per batch, after the batch is reconciled, and costs a few dozen floating-point operations.

4.1 Estimating the cost model

Each completed batch yields one observation (n_k, R_k) of the linear model (2). Writing $x_k = (1, n_k)^\top$ and $\theta = (c_0, c_1)^\top$, we estimate θ with exponentially-weighted recursive least squares:

$$\begin{aligned} K_k &= \frac{P_{k-1}x_k}{\gamma + x_k^\top P_{k-1} x_k},\\ \hat\theta_k &= \hat\theta_{k-1} + K_k\big(R_k - x_k^\top \hat\theta_{k-1}\big),\\ P_k &= \gamma^{-1}\big(P_{k-1} - K_k x_k^\top P_{k-1}\big). \end{aligned} \tag{7}$$

The forgetting factor $\gamma \in (0,1)$ sets the memory; we use $\gamma = 0.995$, an effective horizon of about two hundred batches, or roughly three seconds at the operating point. Because the window itself varies, the regressor n_k is persistently exciting without any injected dither — a convenient side effect of closed-loop operation.

4.2 Estimating load

The arrival rate is estimated by an exponentially weighted moving average of n_k / W_k with smoothing constant $\beta = 0.05$. We considered a Kalman filter with a random-walk prior, and found it no better on our traces at ten times the code.

4.3 The guard

Equation (5) is only meaningful when $\hat\rho_1 = \hat c_1 \hat\lambda < 1$. If load approaches capacity, or if the estimates are briefly nonsense after a regime change, the plug-in window can be wildly wrong. The guard enforces $W_{k+1} \ge (1+\epsilon)\,\hat c_0/(1-\hat\rho_1)$ whenever $\hat\rho_1 < 1-\epsilon$, and otherwise enters *drain mode*: it holds the window at $W_{\max}$ and raises an alert, on the grounds that no window will save a reconciler that cannot keep up with per-event work.

Algorithm 1 Tidewater, one step after batch k

1. **input** batch size n_k, service time R_k, window W_k
2. update $\hat{\theta} = (\hat{c}_0, \hat{c}_1)$ by (7)
3. $\hat{\lambda} \leftarrow (1-\beta)\hat{\lambda} + \beta\, n_k / W_k$
4. $\hat{\rho}_1 \leftarrow \hat{c}_1 \hat{\lambda}$
5. **if** $\hat{\rho}_1 \geq 1 - \epsilon$ **then return** $W_{\max}$ ▷ *drain mode*
6. $W^\star \leftarrow$ equation (5) with $\hat{c}_0, \hat{\rho}_1$
7. $W^\star \leftarrow \max\big(W^\star, (1+\epsilon)\hat{c}_0/(1-\hat{\rho}_1)\big)$ ▷ *guard*
8. $\Delta \leftarrow \mathrm{clip}(W^\star - W_k, -\alpha W_k, \alpha W_k)$ ▷ *rate limit*
9. **return** $\mathrm{clip}(W_k + \Delta, W_{\min}, W_{\max})$

4.4 Rate limiting

The final step moves the window by at most a fraction α of its current value per batch. Without it, a single outlier batch — a garbage-collection pause, say — produces a spike in $\hat{c}_0$ that lengthens the next window, which lengthens the next batch, which the estimator then reads as confirmation. With $\alpha = 0.1$ the loop is well damped and still reaches a new optimum within about twenty batches.

5 Evaluation

5.1 Setup

We replay fourteen days of events from a sandbox settlement environment[2] against a reconciler backed by a single-node balance store. The trace contains 2.9 billion events over 1.84 million distinct accounts, with a diurnal peak-to-trough ratio of about 3.1. We replay it at 1× and at accelerated rates to reach the loads in Table 1. All runs use the same hardware; each configuration is repeated five times and we report the median run.

Parameter	Value
Events replayed	2.91×10^9
Distinct accounts	1,843,206
Arrival rate (replayed)	60–140 k/s
Per-batch overhead c_0 (median)	4.0 ms
Per-event cost c_1 (median)	3.5 μs
RLS forgetting γ / EWMA β	0.995 / 0.05
Rate limit α / guard ε	0.10 / 0.05

Table 1. Workload and controller parameters.

We compare Tidewater against three baselines: Fixed-10 and Fixed-50, timers at 10 and 50 ms, which bracket the windows we found configured in practice; and Size-512, which closes a batch at 512 events with a 100 ms fallback timer. As an upper bound we also report an Oracle that knows the true c_0, c_1 and λ in each one-second interval and applies (5) directly.

5.2 Does the model hold?

Fig. 2 overlays measured mean staleness on the analytic curve. Across all three loads and every stable window, the model is within 6.1% of measurement; the largest errors occur within a millisecond of the stability threshold, where small fluctuations in c_0 push the system briefly into overload. The measured optimum lies within 0.8 ms of $W^\star$ at every load.

5.3 Staleness distributions

Fig. 3 shows the distribution of staleness at 100k events/s. Fixed-10 has the lowest median, because it is closest to per-event reconciliation, but at this load it is only 1.25 ms above its stability threshold and its upper tail is long. Fixed-50 has a short tail and a poor median. Tidewater sits between them in the median and below both in the tail.

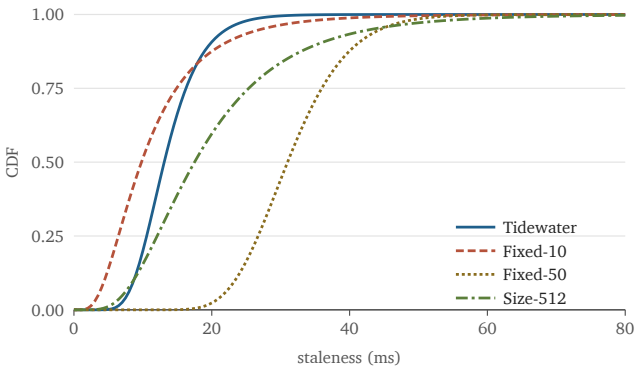


Figure 3. Cumulative distribution of per-event staleness at 100k events/s. Tidewater's distribution is the steepest: it trades a slightly higher median than Fixed-10 for a much shorter tail.

Policy	p50	p99	p99.9	Thr.
Fixed-10	9.8	62.3	188.0	97.1%
Fixed-50	31.0	46.2	58.9	99.6%
Size-512	17.5	63.1	141.7	98.2%
Tidewater	**13.1**	**27.4**	**34.8**	**99.4%**
Oracle	*12.7*	*25.9*	*32.0*	*100%*

Table 2. Staleness quantiles (ms) and throughput relative to Oracle at 100k events/s, median of five runs.

5.4 Responding to drift

The more important test is what happens when conditions change. Fig. 4 shows a two-hour excerpt in which load ramps from 60k to 140k events/s and back while the balance store runs a scheduled compaction that rais-

es c_0 by roughly 40%. Fixed-10 crosses its stability threshold near the peak and its p99 climbs past 130 ms; once it is overloaded, the queue takes several minutes to drain after load falls. Tidewater lengthens its window as $\hat{c}_0$ and $\hat{\lambda}$ rise — from 13 ms to a peak of 31 ms — and its p99 never exceeds 52 ms.

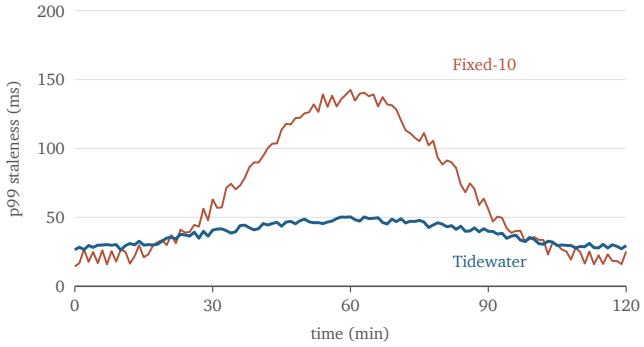


Figure 4. One-minute p99 staleness during a load ramp with concurrent compaction. The fixed window crosses its stability threshold; Tidewater moves its window and stays bounded.

A latency-only controller of the kind proposed in [9] — which shortens the window when observed staleness rises — behaves worse than either policy in this excerpt: rising staleness near saturation causes it to *shorten* the window, which raises overhead and pushes the system further into overload. Proposition 2 explains why. Near saturation, the gradient of staleness with respect to the window points the other way.

5.5 Ablation

Fig. 5 removes one component at a time. Replacing RLS with a fixed, offline-fitted cost model costs the most in the drift excerpt, because the offline model cannot see compaction. Removing the guard produces two brief overload episodes in fourteen days, which dominate the p99. Rate limiting matters less on average but eliminates the self-reinforcing spikes described in §4.4.

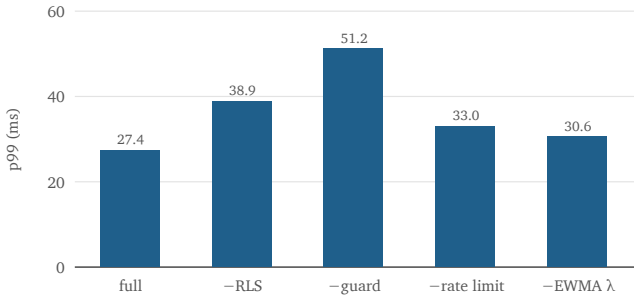


Figure 5. p99 staleness (ms) over the full fourteen-day replay with one component removed at a time.

5.6 Overhead

A Tidewater step costs 41 ns on our hardware, measured over 10^8 steps; at the operating point it runs about seventy times per second. Its memory footprint is a 2×2 covariance matrix, four scalars and a window. We consider this negligible.

5.7 Sensitivity to controller parameters

Tidewater has four parameters. Table 3 varies each in turn around the defaults of Table 1, holding the others fixed, and reports p99 staleness over the full replay. The controller is insensitive to the forgetting factor across an order of magnitude of effective memory and to the guard margin between 2% and 10%. It is most sensitive to the rate limit: at $\alpha = 0.5$ the loop under-damps after each compaction and the p99 rises by a third. We recommend $\alpha \le 0.15$.

Parameter	Low	Default	High
γ (0.98 / 0.995 / 0.999)	28.9	27.4	28.1
β (0.01 / 0.05 / 0.2)	31.7	27.4	29.0
α (0.02 / 0.10 / 0.5)	33.5	27.4	36.6
ϵ (0.02 / 0.05 / 0.10)	28.0	27.4	27.9

Table 3. p99 staleness (ms) over the fourteen-day replay as each parameter is varied.

5.8 How wrong can a fixed window be?

Fig. 6 answers the question operators ask first: if I keep my fixed window, how much am I losing? Each cell gives the ratio of expected staleness at that window and load to the expected staleness at the optimum for that load. A 20 ms window is within 10% of optimal from 60k to 100k events/s and is 47% worse at 140k. A 5 ms window is unstable above 70k. No single row is good everywhere, which is the whole argument of this paper in one picture.

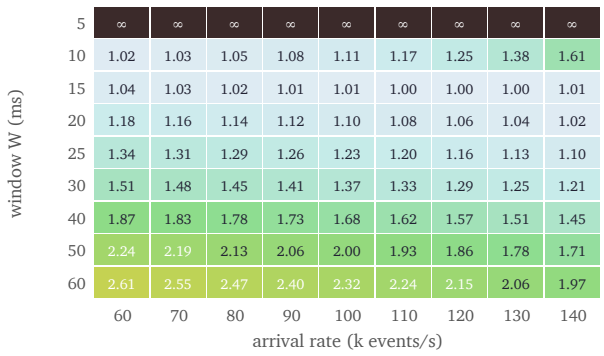


Figure 6. Expected staleness relative to the per-load optimum, from equation (4). Dark cells marked ∞ are unstable. Darker shading means a worse choice; no fixed window is pale across the whole row.

6 Deployment experience

Tidewater has run in the sandbox environment's shadow reconciler for eleven weeks. Shadow mode computes the window Tidewater would choose while the production reconciler continues on its fixed timer, which let us compare decisions without risk. Fig. 7 shows one representative day. The window follows load through the morning rise and the afternoon, lengthens during the scheduled compaction at about 02:00 and the unscheduled one at 13:00, and peaks during the end-of-day cut-over, when both load and overhead are high.

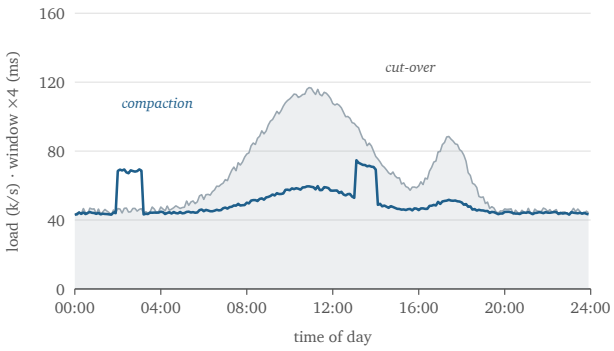


Figure 7. One day of shadow operation. Grey: arrival rate. Blue: the window Tidewater chose, scaled by four to share the axis. The window responds to overhead (compaction) as well as to load (cut-over).

Three lessons from shadow operation surprised us. First, $\hat{c}_0$ is an excellent compaction detector; operators now watch it on a dashboard for that reason alone. Second, the drain-mode alert fired twice in eleven weeks, both times correctly, and both times about forty seconds before the existing queue-depth alert. Third, the most common question we received was why the window got *longer* when the system was busy. Proposition 2 turned out to be a useful thing to be able to point at.

We also learned what not to estimate. An early version fitted a quadratic cost model, on the theory that contention grows faster than linearly in batch size. The quadratic term was never significantly different from zero on our traces, and the extra parameter slowed convergence after compaction by a factor of three. We removed it.

7 Related work

Batching for throughput is as old as group commit [6], and the trade-off between batch size and latency appears in network interrupt coalescing [11], in stream processors' micro-batch intervals [2, 15], and in write-ahead log flushing [7]. Closest to our setting, Okonkwo and Brandt [13] tune micro-batch intervals with a proportional controller on end-to-end latency, and Haldane et al. [9] add an integral term. Both treat the system as a black box. Our contribution is to open the box just far enough — a two-parameter linear cost model — to obtain a closed-form optimum, and to show that this small amount of structure is what lets a controller behave correctly near saturation.

Bounded staleness as a consistency guarantee is well studied in replicated storage [1, 17], where the bound is usually enforced by refusing reads rather than by scheduling writes. The two approaches compose: a reader that demands staleness below a bound benefits from a reconciler that keeps the bound slack. Queueing models of batch service go back to the bulk-service queues of [3]; we use only the simplest of them.

8 Limitations

Our model assumes a single reconciler. Sharded reconcilers with cross-shard events introduce coordination costs that are not linear in batch size, and we have not modelled them. The Poisson assumption is optimistic for bursty sources; on a synthetic workload with arrivals in clusters of mean size eight, the model's error rose to 14%, although Tidewater's ranking among the policies did not change. Finally, our evaluation uses one workload from one environment. We release the controller and our replay harness so that others can test these claims on theirs.

9 Conclusion

A batch window is a decision, not a constant. With a model just rich enough to separate per-batch from per-event cost, the right decision has a closed form, and the parameters it needs can be learned from the batches themselves. The resulting controller is a dozen lines of code, costs nanoseconds per step, and roughly halves tail staleness at every load we tried. We hope it retires a configuration constant or two.

Acknowledgements

We thank the operators of the sandbox environment for their patience with our replay harness, and the anonymous reviewers for a careful reading that improved §3 considerably.

Notes

[1] End-of-day cut-over is the period during which a settlement platform closes one business date and opens the next. Arrival rates spike as merchants flush queued work, and per-batch overhead rises because the balance store is simultaneously writing end-of-day snapshots.

[2] The sandbox contains only synthetic accounts and generated activity; no customer data is involved. Event timing, account popularity and amount distributions were generated to match aggregate statistics rather than individual behaviour.

References

1. A. Brennholt and S. Ivashko. Staleness budgets for geo-replicated reads. In *Proc. Symposium on Replicated Storage*, pp. 112–126, 2019.
2. M. Castellane, R. Odum, and T. Varga. Micro-batching revisited: interval selection in stream engines. *Journal of Data Systems*, 14(3):201–219, 2021.
3. H. Delacroix. On queues served in bulk. *Quarterly Notes in Applied Probability*, 8:44–61, 1961.
4. L. Ferreira-Holt and J. Nakamura. Feedback batching for transaction pipelines. In *Proc. Workshop on Adaptive Systems*, pp. 9–17, 2020.
5. K. Gunnarsdottir. *Ledger Architectures*. Northfold University Press, 2018.
6. P. Halloran. Group commit and its discontents. *Transactions on Storage Engineering*, 3(1):1–22, 1994.
7. D. Hartsfield, E. Moreau, and A. Pell. Flush scheduling for write-ahead logs. In *Proc. Conference on Durable Systems*, pp. 330–342, 2016.
8. T. Iwu and C. Lindqvist. Measuring visibility lag in financial ledgers. *Payments Engineering Review*, 6(2):55–71, 2022.
9. R. Haldane, F. Osei, and N. Petrov. PI control of micro-batch intervals. In *Proc. Symposium on Stream Processing*, pp. 77–89, 2022.
10. S. Jovanovic. Recursive estimation with forgetting: a practitioner's guide. *Control Practice Letters*, 11:150–163, 2017.
11. G. Kerrigan and Y. Sato. Adaptive interrupt moderation. *Networked Systems Quarterly*, 21(4):388–401, 2015.
12. M. Lenkov-Rhee. Persistent excitation in closed-loop identification. *Annals of System Identification*, 30:1–19, 2014.
13. C. Okonkwo and V. Brandt. Latency-targeted batch sizing. In *Proc. Conference on Data Pipelines*, pp. 240–252, 2023.
14. E. Quist. Contended accounts and the cost of hot keys. *Payments Engineering Review*, 7(1):12–29, 2023.
15. J. Rautio and A. Benali. Triggers, timers and the micro-batch. In *Proc. Workshop on Streaming Semantics*, pp. 1–8, 2020.
16. N. Szabó-Kelly. End-of-day processing at scale. *Journal of Settlement Systems*, 2(2):90–104, 2021.
17. W. Tennant and O. Achebe. Probabilistically bounded staleness in practice. In *Proc. Symposium on Replicated Storage*, pp. 201–214, 2020.
18. Y. Vasquez. Convexity arguments for batch scheduling. *Operations Research Notes*, 19:301–310, 2024.

Appendix A Proof of Proposition 2

Let $u = 1 - \rho_1$ and $z = uW - c_0$, so that $W = (z + c_0)/u$ and $1 - \rho(W) = z/W$. The second term of (4) is then

$$\frac{W(c_0 + \rho_1 W)}{z} = \frac{1}{z}\left(\frac{\rho_1 (z + c_0)^2}{u^2} + \frac{c_0 (z + c_0)}{u}\right).$$

Expanding and collecting powers of z,

$$\begin{aligned}\frac{W(c_0 + \rho_1 W)}{z} &= \frac{\rho_1}{u^2}\, z \;+\; \frac{2\rho_1 c_0}{u^2} + \frac{c_0}{u} \;+\; \frac{c_0^2}{z}\left(\frac{\rho_1}{u^2} + \frac{1}{u}\right) \\ &= \frac{\rho_1}{u^2}\, z \;+\; \kappa \;+\; \frac{c_0^2}{u^2}\cdot\frac{1}{z},\end{aligned}$$

where κ does not depend on W and the last step uses $\rho_1 + u = 1$. Since $dz/dW = u$,

$$\frac{d\,\mathbb{E}[S]}{dW} = \frac{1}{2} + \frac{\rho_1}{u} - \frac{c_0^2}{u\, z^2}.$$

Setting this to zero and multiplying through by $2u$ gives $(u + 2\rho_1)\, z^2 = 2c_0^2$, and $u + 2\rho_1 = 1 + \rho_1$. Hence $z^\star = c_0\sqrt{2/(1+\rho_1)}$ and

$$W^\star = \frac{c_0 + z^\star}{u} = \frac{c_0}{1 - \rho_1}\left(1 + \sqrt{\frac{2}{1+\rho_1}}\right).$$

The second derivative, $2c_0^2/z^3 > 0$, confirms a minimum, and uniqueness follows from Lemma 1. $\blacksquare$

Appendix B Results at every load

Load	Policy	p50	p99	Thr.
60k/s	Fixed-10	7.1	14.9	99.8%
	Fixed-50	29.6	44.0	99.9%
	Tidewater	**8.4**	**8.8**	**99.9%**
100k/s	Fixed-10	9.8	62.3	97.1%
	Fixed-50	31.0	46.2	99.6%
	Tidewater	**13.1**	**27.4**	**99.4%**
140k/s	Fixed-10	—	unstable	81.3%
	Fixed-50	36.2	58.7	99.1%
	Tidewater	**21.7**	**34.6**	**98.9%**

Table 4. Staleness (ms) and throughput relative to Oracle at three loads. At 140k/s Fixed-10 is below its stability threshold and its queue grows without bound; we report throughput only.

Appendix C Notation

Symbol	Meaning
λ	event arrival rate
W, W_k	batch window; window of batch k
N_k, n_k	batch size (random; observed)
c_0, c_1	per-batch overhead; per-event cost
$R(n)$, R_k	service time of a batch; observed
$\rho_1 = c_1\lambda$	utilisation from per-event work
$\rho(W)$	total utilisation, equation (3)
$u = 1 - \rho_1$	capacity left for overhead
S_i	staleness of event i, equation (1)
$W^\star$	optimal window, equation (5)
γ, β	RLS forgetting factor; EWMA constant
α, ϵ	rate limit; guard margin

Appendix D Reproducing the figures

Figures 2 and 6 are computed directly from equation (4) with the parameters of Table 1. Figures 3, 4, 5 and 7 are drawn from the median run of each configuration. The replay harness emits one record per batch — window, size, service time, and the controller's estimates — which is sufficient to regenerate every figure and table in this paper. A plotting script and the per-batch records for the drift excerpt of §5.4 accompany the artefact; the full fourteen-day records are 38 GB and are available on request from the authors at the addresses on the first page.